

INTRODUCTION TO MACHINE LEARNING

MIHIR RAO

Contents

1	Basics	2
1.1	Training data and hypotheses	2
1.2	Linear classifiers and their parameters	3
1.3	Why the dot product describes the boundary	4
1.4	Loss: measuring a prediction's cost	5
1.5	Training error and test error	5
1.6	Learning a classifier	6
2	Perceptrons	8
2.1	The classification setup	8
2.2	The perceptron algorithm	8
2.3	What one update accomplishes	9
2.4	Linear separability	10
2.5	Signed distance and margin	10
2.6	The perceptron performance theorem	12
2.7	Including an offset by expanding the features	13
2.8	When the data are not linearly separable	13
2.9	Where classification fits among learning tasks	15

LECTURE 1: BASICS

MACHINE LEARNING uses data to make decisions. Applications include exoplanet detection, health prediction, legal analysis, labor statistics, phishing detection, facial recognition, and satellite-assisted lending. Studying it helps us apply these methods, understand them, and evaluate their limits.

Given examples with known labels, how can we label a new example? We need data, possible prediction rules, a measure of mistakes, and an algorithm to choose a rule.

1.1 Training data and hypotheses

A data point has measured attributes and a label we want to predict. We put the attributes into a vector so that a rule can use them together.

Definition 1.1 [LABELED TRAINING DATA]. For each $i \in \{1, \dots, n\}$, the FEATURE VECTOR is

$$x^{(i)} = (x_1^{(i)}, \dots, x_d^{(i)})^T \in \mathbb{R}^d,$$

and its LABEL is $y^{(i)} \in \{-1, +1\}$. The TRAINING DATA consist of the n labeled examples

$$\mathcal{D}_n = \{(x^{(1)}, y^{(1)}), \dots, (x^{(n)}, y^{(n)})\}.$$

The superscript (i) identifies the example; the subscript identifies a feature. Thus $x_2^{(3)}$ is the second feature of the third example, rather than a power of x_2 . The transpose makes each feature vector a column vector.

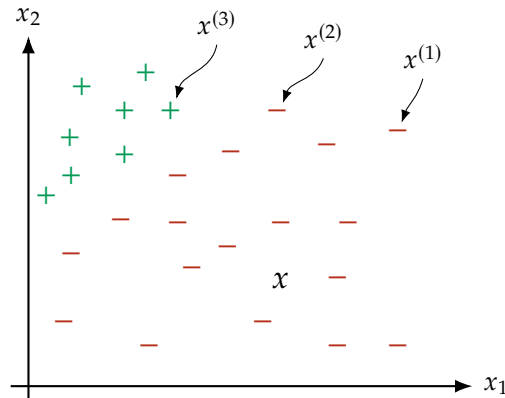


Figure 1: Schematic labeled data in two dimensions. Each position is a feature vector; its + or - gives the observed label. The point marked x still needs a prediction. The coordinates illustrate structure, not measurements.

Definition 1.2 [HYPOTHESIS]. A HYPOTHESIS, or prediction rule, is a function

$$h : \mathbb{R}^d \longrightarrow \{-1, +1\}.$$

Given a feature vector x , it returns a predicted label $h(x)$. In this binary labeling problem, such a rule is also called a CLASSIFIER.

Example: *Always predict +1.* The rule $h(x) = +1$ for every x is a valid hypothesis: it assigns an allowed label to every input. Being a hypothesis does not make it a good one. This rule gets every example with label -1 wrong.

The task is to predict labels for new feature vectors, rather than merely to describe labels already present in $\mathcal{D}_n$.

1.2 Linear classifiers and their parameters

We need to decide which prediction rules the learning algorithm may consider. A HYPOTHESIS CLASS $\mathcal{H}$ is a set of hypotheses. A useful first class assigns $+1$ to one side of a line and -1 to the other. In d dimensions, the corresponding boundary is a hyperplane.

Definition 1.3 [LINEAR CLASSIFIER]. Let $\theta \in \mathbb{R}^d$ and $\theta_0 \in \mathbb{R}$. A LINEAR CLASSIFIER is

$$h(x; \theta, \theta_0) = \text{sign}(\theta^\top x + \theta_0) = \begin{cases} +1, & \theta^\top x + \theta_0 > 0, \\ -1, & \theta^\top x + \theta_0 \leq 0. \end{cases}$$

Here the sign convention assigns -1 when the score is exactly zero. The PARAMETERS are the vector θ and the scalar θ_0 . The class $\mathcal{H}$ contains all rules of this form.

The score $\theta^\top x + \theta_0$ decides the label. When $\theta \neq 0$, the DECISION BOUNDARY is

$$\{x : \theta^\top x + \theta_0 = 0\}.$$

The vector θ sets the boundary's orientation and points toward the positive side. The scalar θ_0 sets its offset. For $\theta = 0$, the score is constant, so the rule predicts one label everywhere; there is no ordinary separating hyperplane. The geometric formulas below assume $\theta \neq 0$.

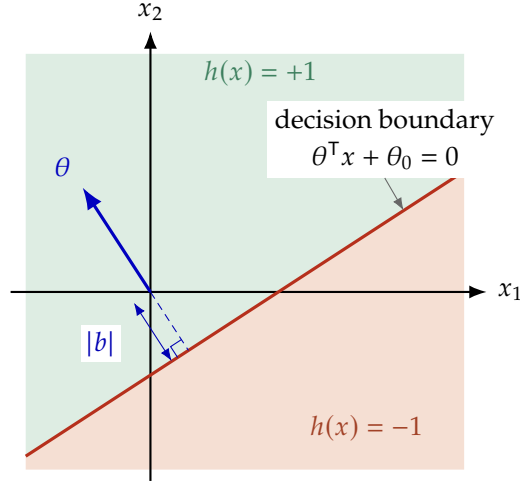


Figure 2: A schematic linear classifier. The normal θ points toward the positive region. The boundary is included in the negative region by the stated sign convention. Its offset is measured along the unit normal.

1.3 Why the dot product describes the boundary

The quantity $\theta^T x$ measures how far x extends in the direction of θ , up to a scale factor. Write

$$u = \frac{\theta}{\|\theta\|}, \quad \|\theta\| = \sqrt{\theta^T \theta}.$$

Since u has length one, the SIGNED PROJECTION of x along this direction is

$$u^T x = \frac{\theta^T x}{\|\theta\|}.$$

Positive values are in the direction of θ ; negative values are in the opposite direction. Dividing the boundary equation by $\|\theta\|$ gives

$$\frac{\theta^T x}{\|\theta\|} = -\frac{\theta_0}{\|\theta\|}.$$

Thus every point on the boundary has the same signed projection. If b denotes that projection, then

$$b = -\frac{\theta_0}{\|\theta\|}, \quad \theta^T x + \theta_0 > 0 \iff u^T x > b.$$

The positive region lies beyond this threshold in the direction of θ . The perpendicular distance from the origin to the boundary is $|b|$; b itself is signed.

For two boundary points p and q , subtract their equations:

$$(\theta^T p + \theta_0) - (\theta^T q + \theta_0) = 0 \implies \theta^T (p - q) = 0.$$

Since $p - q$ runs along the boundary, its zero dot product with θ establishes perpendicularity.

1.4 Loss: measuring a prediction's cost

A classifier should work well on future data. Before comparing classifiers, we must specify what a mistake costs.

Definition 1.4 [Loss]. A LOSS $L(g, a)$ gives the cost of a guessed label g when the actual label is a . For example, the ZERO-ONE LOSS is

$$L(g, a) = \begin{cases} 0, & g = a, \\ 1, & g \neq a. \end{cases}$$

Zero-one loss treats the two kinds of wrong prediction equally. A different task may require different penalties.

Example: *Asymmetric mistakes.* Consider

$$L(g, a) = \begin{cases} 1, & g = +1, a = -1, \\ 100, & g = -1, a = +1, \\ 0, & g = a. \end{cases}$$

Predicting -1 when the actual label is $+1$ costs one hundred times as much as the reverse mistake. A correct prediction costs zero in either class.

The order of the arguments matters: the first is the guess and the second is the actual label. With asymmetric loss, exchanging them changes the cost.

1.5 Training error and test error

To evaluate a classifier on several examples, average its losses. The TRAINING ERROR on the data used for learning is

$$\mathcal{E}_n(h) = \frac{1}{n} \sum_{i=1}^n L(h(x^{(i)}), y^{(i)}).$$

With zero-one loss, the sum counts misclassified examples, so $\mathcal{E}_n(h)$ is the fraction of training examples predicted incorrectly. With asymmetric loss, it is an average cost, rather than a fraction of mistakes.

The TEST ERROR on n' new examples, indexed $n + 1, \dots, n + n'$, is

$$\mathcal{E}(h) = \frac{1}{n'} \sum_{i=n+1}^{n+n'} L(h(x^{(i)}), y^{(i)}).$$

The two formulas use the same loss but different data. The test average measures performance on the new examples, which is the behavior we want to assess.

When choosing from training data, a natural preference is h over $\tilde{h}$ if $\mathcal{E}_n(h) < \mathcal{E}_n(\tilde{h})$. This compares their training performance. It does not establish the same ordering of their test errors.

1.6 Learning a classifier

A classifier takes an input vector and returns a label. A **LEARNING ALGORITHM** has a different input and output: it takes training data and returns a classifier from the chosen class. For linear classifiers, this means choosing θ and θ_0 using $\mathcal{D}_n$.

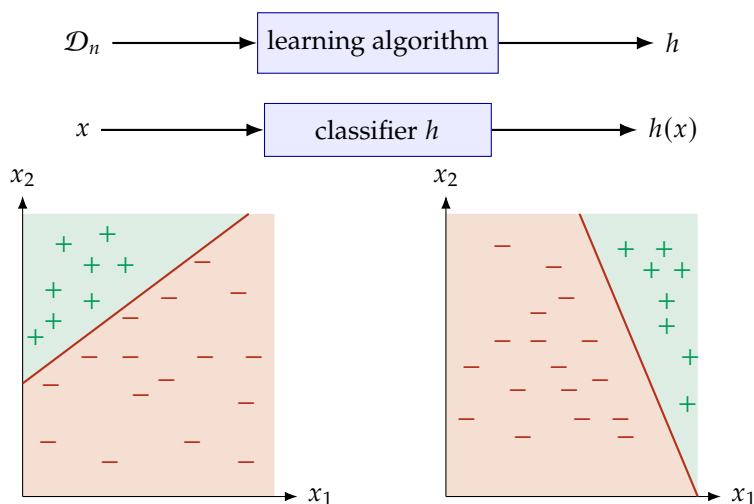


Figure 3: Learning chooses a rule from labeled data; prediction applies that rule to a feature vector. The two schematic data arrangements below admit different separating lines, illustrating why the chosen parameters depend on the training data.

An example: search a random candidate pool

One conceptual learning algorithm constructs many candidate classifiers and selects the one with the smallest training error. Let $M = 10^{12}$, and randomly sample a fixed pool of M parameter pairs. For an integer $1 \leq k < M$, compare only the first k candidates. Write

$$h^{(j)}(x) = h(x; \theta^{(j)}, \theta_0^{(j)}), \quad j = 1, \dots, M.$$

The superscript (j) now indexes candidates, while (i) indexes data points.

Algorithm 1.1: Select a classifier from a random candidate pool

Input: Training data $\mathcal{D}_n$; integer $1 \leq k < M$, where $M = 10^{12}$.

Output: A classifier with smallest training error among the first k candidates.

```
1 for  $j \leftarrow 1$  to  $M$  do
2   | sample a parameter pair  $(\theta^{(j)}, \theta_0^{(j)})$ 
3   | Set  $h^{(j)}(x) \leftarrow h(x; \theta^{(j)}, \theta_0^{(j)})$ 
4   |  $j^* \leftarrow \arg \min_{j \in \{1, \dots, k\}} \mathcal{E}_n(h^{(j)})$ 
5 return  $h^{(j^*)}$ 
```

If several candidates have the same minimum error, choose the smallest index. The data determine which candidate wins within the pool. The number k is a **HYPERPARAMETER**: it controls the learning procedure, rather than appearing in the classifier's prediction formula. The classifier's parameters remain θ and θ_0 .

Proposition 1.5 [COMPARING ONE CANDIDATE WITH TWO]. *For the same data, loss, and fixed candidate pool, the rule selected with $k = 2$ has training error no larger than the rule selected with $k = 1$.*

Proof. The first candidate remains available when the second is added. With $k = 1$, the selected error is $\mathcal{E}_n(h^{(1)})$. With $k = 2$, it is

$$\min\{\mathcal{E}_n(h^{(1)}), \mathcal{E}_n(h^{(2)})\} \leq \mathcal{E}_n(h^{(1)}).$$

It decreases if the second candidate has strictly smaller error; otherwise it stays the same. $\square$

This comparison requires a shared pool, so the first candidate is the same in both runs. It does not compare independent random draws and does not guarantee an improvement in test error.

Revision. Data pair features with labels. A hypothesis predicts a label; a hypothesis class lists allowed rules. Linear classifiers use the sign of $\theta^\top x + \theta_0$. Training and test error average the chosen loss on different data. Learning uses training data to choose classifier parameters.

Common mistakes: confusing example indices with feature indices, assigning +1 on a zero score, swapping guess and actual label in an asymmetric loss, or treating smaller training error as a guarantee of smaller test error.

Check yourself: Why is θ normal to the boundary? How does asymmetric loss distinguish mistakes? Why can adding a candidate never increase minimum training error in a fixed pool? How does k differ from θ and θ_0 ?

Source. Tamara Broderick, MIT 6.036/6.862, *Introduction to Machine Learning*, Fall 2020, [Lecture 1: Basics](#), PDF pp. 34–185 (numbered slides 5–10), via the [official lecture collection](#). Repeated animation builds are consolidated and administration omitted. The geometric steps and the candidate-comparison argument are completed here; the figures are original schematic redraws.

LECTURE 2: PERCEPTRONS

How can we choose a linear classifier by using the labeled examples directly? The perceptron starts with zero parameters, scans the training data, and changes the parameters whenever an example fails a strict separation test. We will see what one update accomplishes and when repeated updates are guaranteed to stop.

2.1 The classification setup

Our data are $\mathcal{D}_n = \{(x^{(i)}, y^{(i)})\}_{i=1}^n$, with $x^{(i)} \in \mathbb{R}^d$ and $y^{(i)} \in \{-1, +1\}$. A linear classifier uses

$$h(x; \theta, \theta_0) = \begin{cases} +1, & \theta^\top x + \theta_0 > 0, \\ -1, & \theta^\top x + \theta_0 \leq 0. \end{cases}$$

The class $\mathcal{H}$ consists of these rules as θ and θ_0 vary. For $\theta \neq 0$, the boundary $\theta^\top x + \theta_0 = 0$ is a hyperplane with normal vector θ .

With zero-one loss, $L(g, a) = 0$ if $g = a$ and 1 otherwise. Thus

$$\mathcal{E}_n(h) = \frac{1}{n} \sum_{i=1}^n L(h(x^{(i)}), y^{(i)})$$

is the fraction of training examples misclassified. Our aim is to find parameters that make this error small.

2.2 The perceptron algorithm

For an example $(x^{(i)}, y^{(i)})$, multiply the classifier's score by its label:

$$y^{(i)}(\theta^\top x^{(i)} + \theta_0).$$

THIS LABEL-WEIGHTED SCORE is positive when the example is strictly on the side prescribed by its label, negative when it is on the wrong side, and zero when it lies on the boundary. A negative label reverses the score's sign, so the same test works for both classes.

THE PERCEPTRON updates whenever this quantity is nonpositive:

$$\theta \leftarrow \theta + y^{(i)}x^{(i)}, \quad \theta_0 \leftarrow \theta_0 + y^{(i)}.$$

Positive examples add $x^{(i)}$ and 1; negative examples subtract $x^{(i)}$ and 1.

Algorithm 2.1: Perceptron with an offset

Input: Labeled data $\mathcal{D}_n$; maximum number of passes $\tau \geq 1$.

Output: Parameters (θ, θ_0) of a linear classifier.

```
1  $\theta \leftarrow \mathbf{0} \in \mathbb{R}^d; \theta_0 \leftarrow 0$ 
2 for  $t \leftarrow 1$  to  $\tau$  do
3   changed  $\leftarrow$  false
4   for  $i \leftarrow 1$  to  $n$  do
5     if  $y^{(i)}(\theta^\top x^{(i)} + \theta_0) \leq 0$  then
6        $\theta \leftarrow \theta + y^{(i)}x^{(i)}$ 
7        $\theta_0 \leftarrow \theta_0 + y^{(i)}$ 
8       changed  $\leftarrow$  true
9   if changed = false then
10    break
11 return  $(\theta, \theta_0)$ 
```

The initial vector has d zeros, one for each feature. Each `PASS` visits all n examples, using the current parameters after any earlier update. The pass budget τ is a hyperparameter of the learning procedure.

Note. The update test includes *every* boundary point. With our sign convention, a boundary point labeled -1 is already predicted correctly, but it still triggers an update because its label-weighted score is zero. Initially every score is zero, so the first visited example triggers an update as well.

If a whole pass makes no changes, the parameters remain fixed throughout it and every example has a positive label-weighted score. All training examples are then classified correctly. Reaching the cap τ is a different stopping condition: it does not by itself establish zero training error.

2.3 What one update accomplishes

An update increases the label-weighted score of the example that caused it. To see this, abbreviate that example to (x, y) and let $\theta' = \theta + yx$ and $\theta'_0 = \theta_0 + y$. Expanding the new score gives

$$\begin{aligned} y((\theta')^\top x + \theta'_0) &= y((\theta + yx)^\top x + \theta_0 + y) \\ &= y(\theta^\top x + \theta_0) + y^2(x^\top x + 1) \\ &= y(\theta^\top x + \theta_0) + \|x\|^2 + 1. \end{aligned}$$

The last step uses $y \in \{-1, +1\}$, hence $y^2 = 1$, and $x^\top x = \|x\|^2$. The $+1$ comes from updating the offset. Since $\|x\|^2 + 1 > 0$, the score strictly increases.

The score can still be nonpositive, and other examples can remain wrong. Repeated scans are therefore needed.

2.4 Linear separability

Before asking whether perceptron can find a perfect classifier, ask whether such a classifier exists in $\mathcal{H}$.

Definition 2.1 [LINEAR SEPARABILITY]. A training set is LINEARLY SEPARABLE if there exist θ and θ_0 such that

$$y^{(i)}(\theta^\top x^{(i)} + \theta_0) > 0 \quad \text{for every } i \in \{1, \dots, n\}.$$

Both classes must be strictly on their respective sides of the same boundary. Positive examples have positive scores; negative examples have negative scores. No training point is allowed on the boundary in this definition.

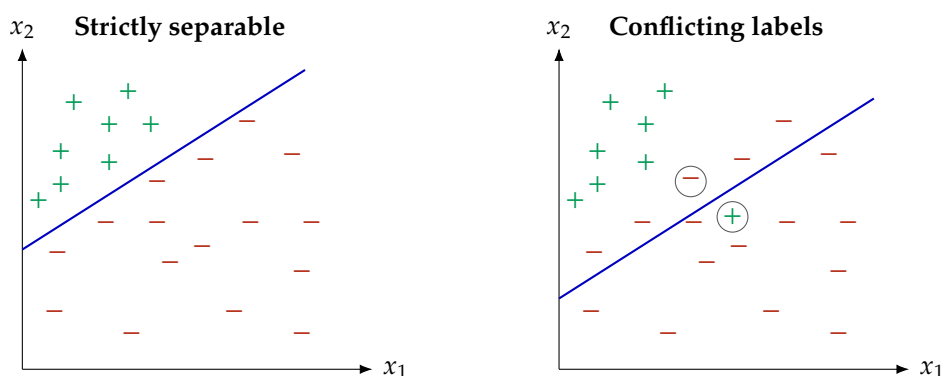


Figure 4: Schematic redraws of two labeled arrangements. The left set admits strict separation. The right set contains conflicting points among the opposite class; reversing the displayed line's orientation does not solve the separation problem. Circles identify the conflicting points.

2.5 Signed distance and margin

Correct separation tells us which side a point occupies. Distance tells us how far it is from the boundary, which lets us distinguish easier separation from separation with points very close to the boundary.

For $\theta \neq 0$, the signed projection of x^* along the unit normal is $\theta^\top x^* / \|\theta\|$. The boundary's signed projection from the origin is $b = -\theta_0 / \|\theta\|$. Subtracting these gives the SIGNED DISTANCE from the boundary to the point:

$$\delta(x^*; \theta, \theta_0) = \frac{\theta^\top x^*}{\|\theta\|} - \left(-\frac{\theta_0}{\|\theta\|} \right) = \frac{\theta^\top x^* + \theta_0}{\|\theta\|}.$$

It is positive on the positive side, negative on the negative side, and zero on the boundary. Its absolute value is the perpendicular distance.

Definition 2.2 [POINT AND TRAINING-SET MARGINS]. The MARGIN of a labeled point (x^*, y^*) with respect to a hyperplane with $\theta \neq 0$ is

$$m(x^*, y^*; \theta, \theta_0) = y^* \frac{\theta^\top x^* + \theta_0}{\|\theta\|}.$$

The margin of the training set with respect to that hyperplane is the smallest point margin:

$$m(\mathcal{D}_n; \theta, \theta_0) = \min_{i \in \{1, \dots, n\}} y^{(i)} \frac{\theta^\top x^{(i)} + \theta_0}{\|\theta\|}.$$

Multiplying by the label makes a correctly separated point's margin positive in either class. A negative-class point can have negative signed distance and positive margin. A point on the wrong side has negative margin; a boundary point has zero margin. The training-set margin checks the least favorable example, so it is positive precisely when this hyperplane strictly separates the whole set.

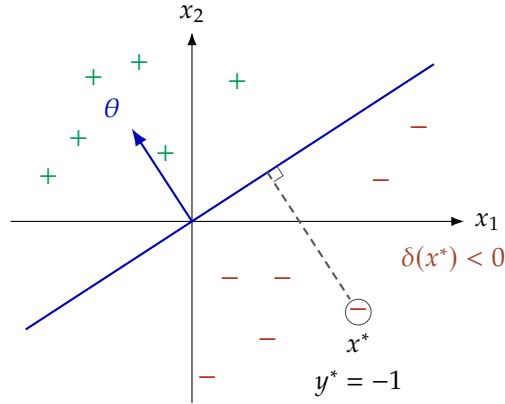


Figure 5: A schematic negative-class point has $\delta(x^*) < 0$ but positive margin $y^* \delta(x^*)$ when $y^* = -1$. The dashed segment is perpendicular to the boundary, rather than a horizontal or vertical distance.

2.6 The perceptron performance theorem

A strictly separating hyperplane with a positive margin gives a bound on the number of perceptron updates. Here we first restrict the classifier to pass through the origin: hold $\theta_0 = 0$ throughout and update only θ .

Theorem 2.3 [PERCEPTRON PERFORMANCE]. *Consider the origin-constrained perceptron, initialized at $\theta = 0$. Assume there exist a nonzero separator θ^* and a number $\gamma > 0$ such that, for every training example,*

$$y^{(i)} \frac{(\theta^*)^\top x^{(i)}}{\|\theta^*\|} > \gamma.$$

Also assume there is an R such that $\|x^{(i)}\| \leq R$ for every i . Then the algorithm makes at most

$$\left(\frac{R}{\gamma}\right)^2$$

updates to θ . A complete pass without an update has zero training error.

The vector θ^* is a fixed separating rule whose existence is assumed; the algorithm does not need to know it. The number γ is a positive lower bound on its point margins. The radius R bounds how far any feature vector lies from the origin.

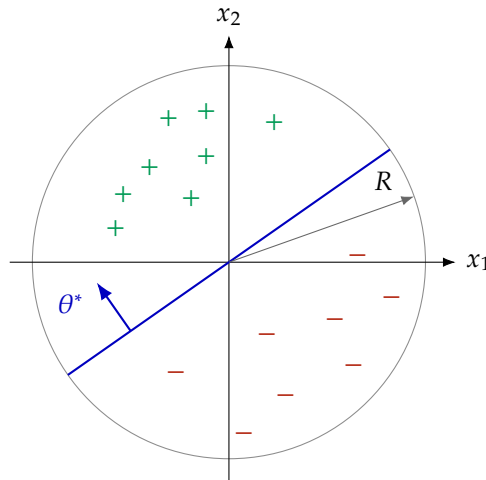


Figure 6: Schematic theorem assumptions: the separating line passes through the origin, all points lie within radius R , and their label-adjusted distances from the line exceed a positive lower bound γ .

The bound counts *updates*, not passes or point visits. A smaller guaranteed margin increases the bound. It guarantees eventual termination on data satisfying the assumptions when enough passes are allowed, but does not promise completion before an arbitrarily small pass cap.

2.7 Including an offset by expanding the features

Restricting a hyperplane to the origin need not restrict the original prediction rule. Add one constant feature and put the offset into its weight:

$$x_{\text{new}} = [x_1 \ \cdots \ x_d \ 1]^\top, \quad \theta_{\text{new}} = [\theta_1 \ \cdots \ \theta_d \ \theta_0]^\top.$$

Both vectors are in $\mathbb{R}^{d+1}$, and

$$\theta_{\text{new}}^\top x_{\text{new}} = \theta^\top x + \theta_0.$$

The scores and predicted labels are unchanged. An offset boundary in the original space becomes a boundary through the origin in the expanded space.

The expanded perceptron update also recovers both original updates:

$$\theta_{\text{new}} \leftarrow \theta_{\text{new}} + y x_{\text{new}} \iff \begin{cases} \theta \leftarrow \theta + yx, \\ \theta_0 \leftarrow \theta_0 + y. \end{cases}$$

Apply the theorem using the radius and margins in the expanded space. In particular, $\|x_{\text{new}}\| = \sqrt{\|x\|^2 + 1}$, and the margin uses $\|\theta_{\text{new}}\| = \sqrt{\|\theta\|^2 + \theta_0^2}$ in its denominator. The original radius and margin therefore cannot simply be reused unchanged.

2.8 When the data are not linearly separable

Real data need not admit strict linear separation. If no separating rule exists in the chosen class, the theorem's positive-margin assumption fails. Repeated perceptron updates then have no convergence guarantee from this theorem, and the pass cap still provides a stopping condition.

Example: *Penguin measurements.* Flipper length and body mass form a two-dimensional feature vector; species supplies a label. The Adelie and Chinstrap groups overlap in these features. A perfect straight boundary should not be assumed merely because the examples have labels.

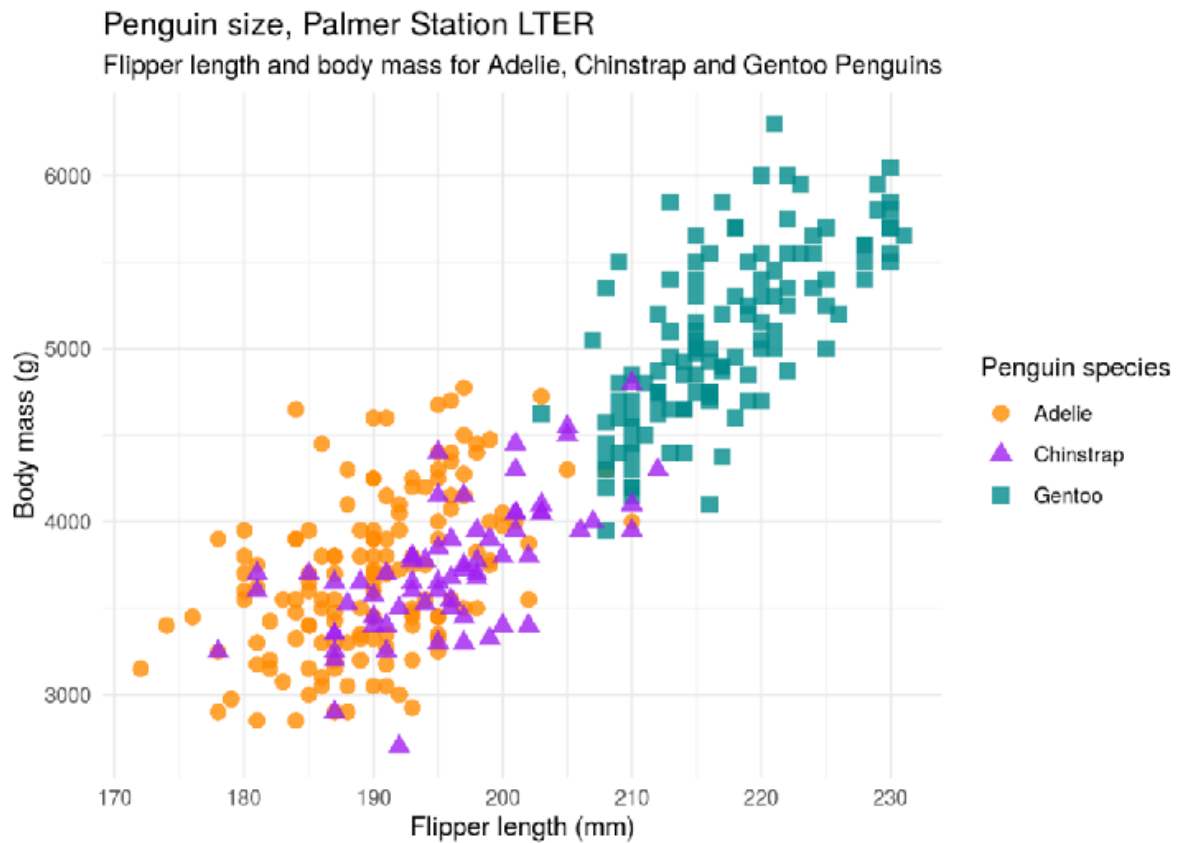


Figure 7: Penguin flipper length and body mass, colored by species. The overlapping groups show why strict linear separation must be checked.

2.9 Where classification fits among learning tasks

Definition: SUPERVISED LEARNING uses labeled examples to learn a mapping from features to labels. UNSUPERVISED LEARNING has no supplied labels and seeks patterns in the data.

Within supervised learning, the output type distinguishes two tasks. REGRESSION predicts continuous values, modeled as a mapping $\mathbb{R}^d \rightarrow \mathbb{R}^k$. Classification predicts a value in a discrete label set. BINARY CLASSIFICATION, or two-class classification, uses two labels, such as $\{-1, +1\}$. MULTICLASS CLASSIFICATION uses more than two label values. Linear classification is one choice of hypothesis class for the binary task.

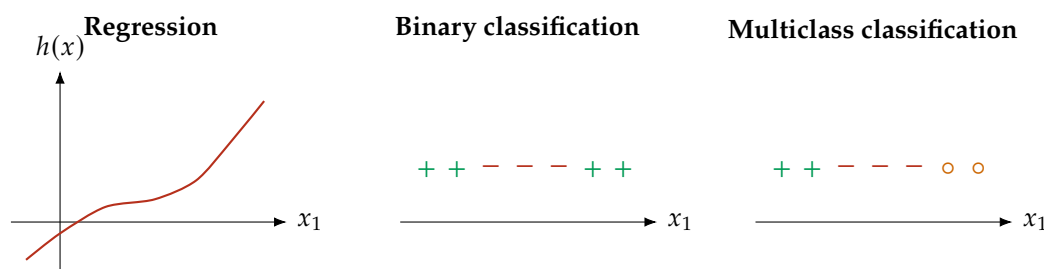


Figure 8: Schematic output types. Regression predicts a continuous value; classification predicts a label. Two symbols represent a binary label set, while three represent a multiclass label set. These arrangements do not assert that a linear rule separates the labels.

Revision. Perceptron updates a point with nonpositive label-weighted score by adding yx to θ and y to the offset. This raises that point's score by $\|x\|^2 + 1$. Strict separation requires positive point margins. With an origin separator of margin greater than γ and feature norms at most R , the update bound is $(R/\gamma)^2$. A constant feature incorporates an offset into an origin-constrained rule in one higher dimension.

Common mistakes: skipping negative-labeled boundary points, confusing score with normalized margin, counting passes as updates, or assuming the theorem applies to nonseparable data or guarantees performance on new data.

Check yourself: Why does the offset contribute $+1$ to the update calculation? Why is a negative-class point's margin positive on the correct side? What does a no-change pass establish? Which norms change when the constant feature is added?

Source. Tamara Broderick, MIT 6.036/6.862, *Introduction to Machine Learning*, Fall 2020, [Lecture 2: Perceptrons](#), PDF pp. 2–157 (numbered slides 2–9), via the [official lecture collection](#). Animation builds are consolidated. The update algebra and feature-expansion identities are completed; the fixed separator's norm is written consistently as $\|\theta^*\|$ in the theorem. The diagrams are original schematic redraws. The penguin figure is reproduced from PDF p. 138; data: [Palmer Penguins project](#).